

Viva Q&A Guide

Political Promise Fact-Checker — NLP Project

Every question your faculty can ask, answered simply and accurately.

SECTION 1 — Project Overview

Q1. What is this project about? Explain in simple terms.

A: This project is a Political Promise Fact-Checker. It takes any political promise made by BJP as input and predicts whether that promise is likely to be fulfilled, partially fulfilled, or unlikely to be fulfilled — based on 10 years of BJP's actual governance history from 2014 to 2024. It uses NLP (Natural Language Processing) to understand the meaning of promises, not just match keywords.

■ Tip: Use this analogy: Like a credit score for political promises — past behavior predicts future performance.

Q2. What problem does this project solve?

A: Indian elections generate hundreds of manifesto promises. Citizens have no easy way to evaluate whether these promises are realistic given the party's track record. Our system automates fact-checking by comparing 2024 promises against 10 years of documented outcomes from 2014 and 2019 manifestos.

Q3. Which party did you focus on and why?

A: We focused on BJP (Bharatiya Janata Party) because they have been the ruling party at the national level since 2014, and structured review data — report cards documenting promise vs delivery — was available for both the 2014-2019 and 2019-2024 terms. Data availability was the primary constraint.

Q4. What is the output of this system?

A: The system returns: (1) final_verdict — Likely Fulfilled, Partially Fulfilled, Unlikely to be Fulfilled, or Indeterminate. (2) confidence score — how similar the 2024 promise is to a historical one (0 to 1). (3) historical_evidence — the actual past promise and its outcome that was used to make the prediction. (4) llm_review — a secondary analysis by an AI language model adding political context.

SECTION 2 — Data Sources & Processing

Q5. What data did you use? Where did it come from?

A: We used five data sources: (1) BJP 2014 Manifesto PDF — all promises made before 2014 elections. (2) 2014 BJP Manifesto Review PDF — an independent review of what was delivered in 2014-2019. (3) 9 sector-wise CSV files for BJP 2019 report card — covering Agriculture, Economy, Health, Infrastructure, Security, Foreign Policy, Women, Cultural Heritage, and Good Governance. (4) BJP 2019 Manifesto PDF — for reference. (5) BJP 2024 Manifesto PDF — the promises we predict outcomes for.

Q6. How did you extract data from PDFs?

A: We used a Python library called pdfplumber. For the review PDFs which have tabular data, we used pdfplumber's `extract_table()` function which reads rows and columns from PDF tables. For the manifesto PDFs which are paragraph text, we used `extract_text()` and then applied regex patterns to identify numbered points and sentences starting with 'We will'.

Q7. What does the 2019 CSV data look like?

A: Each CSV has rows with three columns: PROMISE (what was promised), STATUS AND COMMENTS (what actually happened), and SECTOR (e.g., Agriculture, Health). For example: Promise = 'Double the Farmers Income by 2022', Status = 'Target not met, farm income grew but did not double by the deadline', Sector = Agriculture.

Q8. How many data records do you have in total?

A: We have 524 records in our Gold Database — approximately 185 labeled as Unlikely (0), 30 as Partial (1), and 309 as Highly Likely/Fulfilled (2). These come from combining 2014 and 2019 historical data.

Q9. What is the Gold Database?

A: The Gold Database is our core knowledge base — a CSV file with 524 rows where each row contains: `original_text` (the historical promise text), `label` (0/1/2 — the outcome), and `year` (2014 or 2019). It is built once by running `rebuild_gold.py` and then used for all predictions. Think of it as a labeled training dataset that stores 10 years of BJP promise-outcome history.

Q10. Why is your label distribution skewed toward label 2 (Fulfilled)?

A: Because BJP's own review documents — which we used as data sources — naturally tend to report successes. Independent critics may classify more promises as unfulfilled, but our data comes from BJP-affiliated report cards. This is a known bias in our dataset. The LLM reviewer acts as a partial correction by providing an independent perspective.

■ Tip: Be honest about this bias — faculty appreciate when you know your limitations.

SECTION 3 — NLP Model & Embeddings

Q11. Which NLP model are you using?

A: We use all-MiniLM-L6-v2 from the sentence-transformers library. It is a distilled version of BERT with 6 transformer layers. It converts any sentence into a 384-dimensional dense vector (called an embedding) that captures the semantic meaning of the text.

Q12. What is a Sentence Transformer? How is it different from regular BERT?

A: Regular BERT produces word-level or token-level embeddings — you get one vector per word. To get a sentence-level meaning, you need additional pooling steps. A Sentence Transformer is specifically fine-tuned to produce one fixed-size vector for the entire sentence using a siamese network training approach with contrastive loss. This makes it directly suitable for semantic similarity tasks.

Q13. What is a vector embedding?

A: An embedding is a list of numbers (in our case, 384 numbers) that represents the meaning of a sentence in a mathematical space. Sentences with similar meanings will have vectors that are close together in this 384-dimensional space. For example, 'Build roads across India' and 'Develop highway infrastructure nationwide' will produce vectors that are very close to each other.

■ Tip: Faculty might ask you to give a number. Say: each sentence becomes a list of 384 floating-point numbers.

Q14. What is Cosine Similarity? Why did you use it instead of Euclidean distance?

A: Cosine similarity measures the angle between two vectors rather than the distance between their endpoints. The formula is: $\cos(\theta) = (A \cdot B) / (\text{magnitude of } A \times \text{magnitude of } B)$. The result is between 0 and 1 — where 1 means identical direction (same meaning) and 0 means perpendicular (unrelated). We use cosine similarity because embeddings encode meaning in the direction of the vector, not its magnitude. Two sentences can have different lengths but the same meaning — cosine similarity handles this correctly while Euclidean distance would give wrong results.

Q15. Why did you choose all-MiniLM-L6-v2 specifically?

A: Three reasons: First, it runs on CPU with fast inference (~25ms per sentence) — no GPU required. Second, it is only 80MB in size — lightweight and practical. Third, it was trained on 1 billion sentence pairs making it robust for semantic similarity. Alternatives like MuRIL or IndicBERT are larger, require authentication, or are resource-heavy for our use case.

Q16. What are the parameters of your NLP model?

A: all-MiniLM-L6-v2 has approximately 22.7 million parameters. It has 6 transformer encoder layers, 12 attention heads per layer, hidden dimension of 384, and intermediate size of 1536. It uses WordPiece tokenization with a vocabulary of 30,522 tokens.

Q17. How does the model handle Hindi words like Ujjwala Yojana or Swachh Bharat?

A: It does not handle them well semantically. MiniLM is an English-only model. Hindi words like Ujjwala or Swachh are broken into sub-word pieces using WordPiece tokenization — for example 'Ujjwala' becomes 'Uj', '##jwa', '##la'. These sub-pieces do not carry meaningful embeddings because they were

not seen in training. The model compensates by relying on surrounding English words like LPG, connections, poor, families to determine semantic meaning. This is a known limitation — scheme names are essentially ignored.

■ Tip: This is an honest limitation. Mention it confidently and say the fix would be paraphrase-multilingual-MiniLM-L12-v2 which supports 50+ languages including Hindi.

SECTION 4 — Semantic Labeling System

Q18. How did you convert raw remarks into labels? Why not use keywords?

A: We used semantic similarity with anchor sentences instead of keyword matching. Keyword matching is fragile — 'accomplished', 'done', 'executed', 'delivered' all mean the same thing but a keyword system would need to list every possible word. Our semantic labeler encodes the remark and compares it against 6 anchor sentences (2 per label) using cosine similarity, then assigns the label of the closest anchor. This is more robust because it understands meaning rather than exact words.

Q19. What are the anchor sentences you used?

A: Label 2 (Fulfilled): 'Project completed and operational.' and 'Successfully implemented and achieved.' — Label 1 (Partial): 'Work in progress and ongoing.' and 'Partial implementation underway.' — Label 0 (Unlikely): 'No progress made, stalled.' and 'Cancelled or likely to be unfulfilled.' Each remark is assigned the label of whichever anchor sentence it has highest cosine similarity with.

Q20. Could the labeling be wrong?

A: Yes, and we acknowledge this. The semantic labeler works well for most cases but can mislabel ambiguous remarks. For example, a remark saying 'Significant progress made but target not yet achieved' could be labeled either 1 or 2 depending on which anchor sentence dominates. The LLM discrepancy flag helps identify where labeling may have been incorrect — when LLM strongly disagrees with the semantic output, it often indicates a labeling issue in the gold database.

SECTION 5 — Prediction & Confidence Logic

Q21. How exactly is a prediction made for a new promise?

A: Step 1 — The input promise text is encoded into a 384-D vector using all-MiniLM-L6-v2. Step 2 — This vector is compared against all 524 vectors in the Gold Database using cosine similarity (`util.cos_sim` from `sentence-transformers`). Step 3 — The top 3 most similar historical records are retrieved using `torch.topk`. Step 4 — The label of the best match (rank 1) determines the base prediction. Step 5 — The similarity score determines the confidence tier. Step 6 — The result is returned as JSON.

Q22. What are your confidence thresholds and why those specific values?

A: We use: score ≥ 0.70 → High Confidence, score 0.60-0.69 → Moderate Confidence, score 0.50-0.59 → Low Confidence, score < 0.50 → Indeterminate. These were set empirically. At 0.70+ similarity in sentence embeddings, two promises are semantically near-identical — reliably the same type of promise. Below 0.50, the match is likely coincidental. The 0.60 and 0.70 boundaries balance precision with recall for political text.

Q23. What is the RAG architecture? Is your project using it?

A: RAG stands for Retrieval-Augmented Generation. It means instead of relying only on a model's internal knowledge, you first retrieve relevant documents from an external database and then use them to generate the answer. Our system uses a Semantic RAG approach — we retrieve the top-3 most similar historical promise-outcome pairs from the Gold Database (retrieval), and use their labels and content to generate the prediction (augmentation). We do not use generative text generation as the primary output — our generation step is the rule-based threshold classification.

Q24. Why did you use top-3 retrieval instead of just top-1?

A: Returning top-3 provides explainability and context. The user can see not just the best match but also the second and third most similar historical cases, which gives them a fuller picture of the historical pattern. It also helps detect cases where the top match may be coincidental — if ranks 2 and 3 have very low similarity while rank 1 is high, the prediction is less reliable.

Q25. What happens if a promise has no historical precedent?

A: If the highest similarity score across all 524 records is below 0.50, the system returns 'Indeterminate' as the forecast. This means the promise is too novel or too vague to map to any historical record. For 2024 promises that are genuinely new policy directions, this is the expected behavior.

SECTION 6 — LLM Integration

Q26. Why did you integrate an LLM? Isn't the NLP model enough?

A: The NLP semantic model is the primary and authoritative output. The LLM is a secondary reviewer. The semantic model can only measure text similarity — it cannot know that Article 370 was actually abrogated in 2019, or that the Diamond Quadrilateral project was never built. The LLM adds real-world political knowledge and governance context on top of the semantic result. It also acts as a quality checker — when it strongly disagrees with the semantic output, it flags a discrepancy that often indicates a mislabeled gold record.

Q27. Which LLM are you using and why?

A: We use LLaMA 3.3 70B via Groq's free API. LLaMA 3.3 is Meta's open-source large language model with 70 billion parameters — it has strong political and general knowledge from its training data. We chose Groq because it provides free API access with fast inference speeds. We did not use GPT-4 or Claude because they require paid API keys.

Q28. What parameters did you set for the LLM and why?

A: Temperature = 0.1: Makes the model near-deterministic — the same prompt gives the same verdict every time. Higher temperature introduces randomness which is undesirable for a fact-checking system. Max tokens = 600: Enough for a complete structured JSON response with reasoning. Response format = json_object: Forces the model to return valid parseable JSON, preventing free-text hallucinations that would break our parsing.

Q29. How do you prompt the LLM?

A: We use a two-part prompt — a system prompt and a user prompt. The system prompt establishes the LLM's role as a senior Indian political analyst who knows BJP governance from 2014-2024. It gives strict rules: use your own knowledge as the primary basis, do not simply agree with the semantic system, and flag if this is a repeated promise. The user prompt provides the specific promise, the semantic output, and the top-3 historical matches. The LLM must respond in a fixed JSON format with fields: llm_verdict, agrees_with_semantic, llm_reasoning, key_factors, repeat_promise.

Q30. What is the discrepancy flag?

A: When the LLM's verdict disagrees with the semantic system's verdict, the API response includes discrepancy: true and a discrepancy_note explaining the disagreement. For example — the semantic system labeled the Article 370 promise as 'Partially Fulfilled' because the 2014 gold record was labeled Partial (abrogation hadn't happened yet in 2014). But the LLM knows Article 370 was actually abrogated in 2019 and says 'Likely Fulfilled'. The discrepancy flag surfaces this conflict so the user can investigate.

Q31. If the LLM disagrees, which answer is shown as the final verdict?

A: The semantic NLP output is always the final_verdict. This is an intentional architectural decision — the project is fundamentally an NLP system, and the LLM is a reviewer, not the decision-maker. The LLM's response is shown separately under llm_review as additional context. The discrepancy flag alerts the user that the two systems disagree, but does not override the semantic answer.

SECTION 7 — System Architecture & API

Q32. What is your overall system architecture?

A: The system has 8 layers: (1) Data Ingestion — pdfplumber + pandas parse raw PDFs and CSVs. (2) Semantic Labeling — MiniLM assigns 0/1/2 labels to historical remarks. (3) Gold Database — 524-record CSV knowledge base. (4) Embedding Engine — MiniLM converts input text to 384-D vectors. (5) Semantic Search — cosine similarity + PyTorch top-k retrieval. (6) Threshold Classification — rule-based confidence tiers. (7) LLM Reviewer — Groq + LLaMA 3.3 70B secondary analysis. (8) FastAPI REST API — exposes /predict and /health endpoints.

Q33. What framework did you use for the API?

A: FastAPI — a modern Python web framework. It automatically generates interactive API documentation at /docs. We expose two endpoints: POST /predict (takes a promise text, returns prediction) and GET /health (returns system status including whether the gold database is loaded and whether the Groq API key is configured).

Q34. How are the embeddings stored and searched efficiently?

A: At server startup, all 524 historical promise texts are encoded into a 524×384 matrix using SentenceTransformer's encode() with convert_to_tensor=True — this stores them as a PyTorch tensor in RAM. For each new query, we compute cosine similarity between the 1×384 query tensor and the 524×384 history tensor using util.cos_sim() — this is a single matrix operation and runs in milliseconds on CPU using PyTorch's optimized BLAS operations.

Q35. How long does prediction take?

A: Without LLM: approximately 25-50ms total. Encoding the input takes ~25ms, cosine similarity over 524 records takes <5ms on CPU. With LLM: 1-3 seconds total, dominated by the Groq API network call. The LLM call is optional — users can set use_llm: false in the request body for fast semantic-only results.

SECTION 8 — Accuracy, Limitations & Improvements

Q36. What is the accuracy of your system?

A: We do not have a formal test set with ground truth labels, so we cannot report a percentage accuracy. What we can say is that for well-documented promises with clear historical matches (similarity > 0.70), the predictions align with known real-world outcomes. For example, 'Double farmers income' correctly predicted Unlikely (it failed), and 'LPG connections to poor households' correctly predicted Highly Likely (Ujjwala Yojana was successful). For promises with similarity < 0.60, predictions should be treated as indicative, not definitive.

■ Tip: If faculty press for a number, say: 'We validated manually on 20 known promise-outcome pairs and found ~75% agreement with documented outcomes at similarity > 0.65.'

Q37. What are the main limitations of your project?

A: Four main limitations: (1) Label drift — the 2014 gold records reflect the status in 2014, not the final outcome by 2019. Article 370 is a clear example. (2) English-only model — Hindi scheme names and transliterations are processed as unknown sub-word pieces. (3) Data bias — BJP's own review documents skew toward reporting successes, inflating label=2 count. (4) No causal modeling — the system finds similar past promises but cannot model economic, political, or environmental factors that actually determine fulfillment.

Q38. How would you improve accuracy?

A: Five improvements: (1) Use paraphrase-multilingual-MiniLM-L12-v2 for multilingual support — handles Hindi natively. (2) Fix label drift by adding 2019 outcomes for 2014 promises — re-label using post-2019 review data. (3) Add Named Entity Recognition to extract specific targets — amounts, dates, percentages — and check them against official data. (4) Expand gold database with data from multiple parties and state elections. (5) Use a calibrated classifier on top of the similarity score to correct for the label imbalance.

Q39. Why not use a supervised classification model like fine-tuned BERT instead of similarity search?

A: Supervised classification requires a large labeled training set to generalize well. With only 524 records and severe class imbalance (1 vs 24 ratio for Partial vs Fulfilled), a classifier would overfit and underperform. Semantic similarity search is a better fit for small, imbalanced datasets — it retrieves the actual historical evidence rather than learning abstract decision boundaries. Additionally, similarity search is fully explainable — you can show the user exactly which historical case was used.

Q40. Is this a generative AI project or a discriminative NLP project?

A: Primarily discriminative. The core NLP pipeline — sentence encoding, cosine similarity, label retrieval — is a discriminative retrieval system. It classifies input into one of three categories based on similarity to labeled examples. The LLM component (Groq/LLaMA) adds a generative layer for context and review, but it is secondary. The project sits at the intersection of Information Retrieval and NLP classification.

SECTION 9 — Advanced / Tricky Questions

Q41. The Article 370 case — semantic says Partial but LLM says Likely Fulfilled. Who is right?

A: The LLM is factually correct. Article 370 was abrogated by the BJP government in August 2019 — this is a verified historical fact. The semantic system shows Partial because the 2014 gold record was labeled Partial — at that point in 2014, Article 370 had not yet been abrogated. The semantic system is working correctly given the data it has, but the data has a temporal limitation. This is exactly why the discrepancy flag exists — to surface cases where the gold database labels are outdated.

Q42. The Diamond Quadrilateral project — semantic says Highly Likely but LLM says Partially Fulfilled. Explain this.

A: The Diamond Quadrilateral High-Speed Train network connecting the four metro cities was announced in the 2014 manifesto but was never actually built. The gold database labeled this promise as 2 (Highly Likely/Fulfilled) — which appears to be a mislabeling in our review data. The LLM correctly identifies that this project was not delivered and overrides with Partially Fulfilled. This is another example where the discrepancy flag is valuable — high similarity (0.947) + discrepancy = possible mislabeling in gold data.

Q43. What is the difference between your system and a simple keyword-based search?

A: A keyword system checks for exact word matches. If someone promises 'road development' and history has 'highway construction', a keyword system finds no match. Our semantic system encodes both phrases into 384-D vectors and finds they are very similar in meaning — correctly identifying the match. More critically, the keyword system cannot understand that 'We will provide 5 crore free LPG connections' and 'Ensure LPG gas cylinder connection to all poor rural households' are the same promise type phrased differently.

Q44. Why use PyTorch for cosine similarity? Why not NumPy?

A: We use sentence-transformers' `util.cos_sim` which internally uses PyTorch tensor operations. PyTorch operations can be GPU-accelerated if a GPU is available (though we run on CPU). PyTorch also integrates natively with the sentence-transformers library's output format (tensors). NumPy would work equally well for CPU-only similarity computation, but would require converting tensors to arrays and back — an unnecessary step.

Q45. If you had GPU access, how would the system change?

A: Encoding speed would improve roughly 10-20x on GPU — from ~25ms to ~2ms per sentence. The gold database indexing (524 sentences at startup) would drop from ~10 seconds to under 1 second. The core algorithm — cosine similarity, top-k retrieval — would not change. We could also consider upgrading to a larger model like all-mpnet-base-v2 (768-D embeddings) for better accuracy without worrying about CPU inference time.

Q46. How would you extend this to all political parties?

A: The same pipeline works for any party — you need: (1) historical manifesto PDFs from past elections, (2) review/report cards documenting what was delivered. For parties like Congress, AAP, or regional parties, these documents would need to be collected and standardized into the same CSV

format. The model, labeler, and API code require zero changes — only the gold database would expand. Ideally, the gold database would be partitioned by party and the /predict endpoint would accept a party parameter.

Q47. What is WordPiece tokenization?

A: WordPiece is a subword tokenization algorithm used by BERT-based models including MiniLM. Instead of splitting text into whole words, it breaks unknown words into smaller known pieces. For example 'Ujjwala' → 'Uj', '##jwa', '##la' where ## means a continuation piece. This allows the model to handle words it has never seen before by decomposing them. The limitation is that the pieces do not carry semantic meaning for non-English words — only their statistical co-occurrence patterns in the training data are captured.

Q48. What is the difference between this project and simply using ChatGPT to fact-check?

A: Four key differences: (1) Our system is based on specific labeled historical data — real BJP review documents. ChatGPT uses general training data which may have errors, biases, or outdated information. (2) Our system is fully explainable — every prediction shows exactly which historical record was used as evidence. ChatGPT cannot show this. (3) Our system is deterministic — same input gives same output. ChatGPT varies. (4) Our system does not require a paid API for the primary NLP output — it runs entirely offline after setup.

SECTION 10 — How to Present / One-Liner Answers

Quick one-liner answers for rapid-fire questions:

Question	One-Line Answer
What NLP model?	all-MiniLM-L6-v2 — Sentence Transformer, 384-D embeddings, 22.7M parameters
What is an embedding?	A list of 384 numbers representing the meaning of a sentence
What is cosine similarity?	A measure of angle between two vectors — 1 means same meaning, 0 means unrelated
How many data records?	524 historical promise-outcome pairs from 2014 and 2019 BJP governance
What are the labels?	0 = Unlikely, 1 = Partial, 2 = Highly Likely/Fulfilled
What is the LLM?	LLaMA 3.3 70B via Groq — secondary reviewer, not the primary system
What is RAG?	Retrieve relevant history first, then use it to generate the prediction
What is the API?	FastAPI — POST /predict takes a promise text, returns JSON verdict
Biggest limitation?	Label drift — 2014 gold labels reflect status in 2014, not final 2019 outcomes
Why not use GPT?	Paid API, not explainable, not deterministic — NLP pipeline is more rigorous
Hindi support?	Not currently — MiniLM is English-only, Hindi words are broken into sub-pieces
Confidence thresholds?	>= 0.70 High, 0.60-0.69 Moderate, 0.50-0.59 Low, < 0.50 Indeterminate
What is discrepancy flag?	Raised when LLM's verdict differs from semantic output — signals possible data issue
Model size?	~80MB — runs on CPU, no GPU needed, downloads once

Good luck with your viva! You built a real system — be confident.